AEGIS

Master Document

Offline, Evidence-First Application Security & Cryptographic Risk Platform

Deterministic security tools establish the facts. A local reasoning agent investigates, correlates and explains them. Neither the source code nor the target application's data leaves the machine.

This document consolidates	
Complete solution	Problem, solution, operating modes, architecture, technology stack, workers, correlation, risk, evidence model.
Visual flow	The full end-to-end flow diagram of every component and surface.
Surfaces specification	What the CLI, Web UI, MCP server and CI/CD gate each provide.
Capabilities	Click-to-fix & verify loop, findings chatbot, ecosystem integrations, OWASP Top 10 coverage.
Delivery	Build scope, differentiation, limitations, worked walkthrough and team build plan.

> **One-line pitch.** AEGIS confirms a weakness on the running app, locates the exact code responsible, maps the surrounding cryptography, ranks it by real-world risk, drafts the fix and re-tests it — entirely offline.

Core principle: deterministic tools establish facts; the local model reasons over them and may never create a finding.

Contents

Section	Covers
1–2 Problem & Solution	The four gaps, the trust gap, the solution and design principles.
2A Operating Modes	Mode 1 / 2 / 3, code ingestion (folder vs repo clone).
3 Architecture + Visual Flow	Component diagram and the full rendered flow.
3A Technology Stack	Every tool, its purpose, and its offline status.
5 Deep Agent	What it may and may not do; the typed decision and evidence contract.
6–8 Workers & Crypto Engine	Runtime, code, and cryptography analysis.
9–11 Correlation, Risk, Evidence	The differentiator, the risk formula, the state machine.
12 Walkthrough	One finding carried end to end.
12B Click-to-Fix & Verify	Three-stage copy → verify → commit, with examples.
12C Findings Chatbot	Hybrid retrieval over findings.
12D Ecosystem	MCP, CI/CD gate, attack-path graph, SARIF.
12E Surfaces Specification	CLI, Web UI, MCP, CI/CD in detail.
12A OWASP Top 10 Coverage	Owner, signal and confidence per category.
13–15 Scope, Differentiation, Limits	Honest build scope and positioning.
16 Team Build Plan	Two-person split and build order.

1. Problem

Application security is spread across runtime scanners, static analyzers, dependency tools, secret scanners and crypto scanners. Each is useful alone; none of the output is connected. That leaves four gaps:

- **Runtime gap.** A dynamic scanner can show an app behaves insecurely but cannot say which code is responsible.
- **Source gap.** A static analyzer can flag a risky pattern but cannot show it actually matters in the deployed app.
- **Cryptography is disconnected.** Weak algorithms, keys and certificates are reported apart from AppSec findings, with no inventory (CBOM).
- **Decision overload.** Teams receive long alert lists instead of one ranked view of impact, root cause and fix.

A fifth, practical gap is **trust in AI tooling**: most AI security agents send source and target data to a cloud model and let the model decide what counts as a finding. Security teams can accept neither.

2. Solution

AEGIS adapts to the evidence it is given. A URL enables runtime analysis; a repository enables code and crypto analysis; both together enable cross-layer correlation — the core value.

Component	Role
Worker 1 — Runtime	Observes the deployed app through controlled, authorized interaction and records reproducible request/response evidence.
Worker 2 — Code	Repository inventory, route extraction, static analysis with data-flow tracking, dependencies, secrets, IaC; file/function/line evidence.
Engine 3 — Crypto	Crypto discovery, CycloneDX CBOM, classical and quantum risk, migration guidance. An *engine* because it is a fixed deterministic pipeline with no decisions of its own.
Deep Agent	Plans, delegates, chooses the next investigation step, explains, prioritizes, drafts remediation and triggers verification.

Design principles

- **Evidence-first.** Only deterministic tools create evidence; the model never creates a finding.
- **Offline-first.** Every component runs locally; an optional connected mode exists only for feed updates and secret verification.
- **Safety outside the model.** Scope, rate limits, test inputs and tool permissions are enforced in code, not prompts.
- **Constrained reasoning.** Agent decisions are typed choices from fixed enums, validated before use.
- **Honest output.** Every claim carries a state — candidate, corroborated, confirmed or resolved — and evidence IDs.

2A. Operating Modes

The input router selects the mode deterministically from what the user supplies. This is the top-level behavior of the whole system.

Mode	User gives	What runs	Output
Mode 1 Runtime	URL only	Worker 1	All runtime-provable vulnerabilities on the live app, each with reproducible request/response evidence.
Mode 2 Code + Crypto	Repository link *or* local folder path	Worker 2 + Engine 3	Code issues, each explained to the user: the vulnerability, the exact file:line, its impact, how it could be exploited, and how to fix it. Plus the crypto inventory (CBOM).
Mode 3 Correlated	URL + (repository or folder)	Worker 1 + Worker 2 + Engine 3, then correlation	Everything from Modes 1 and 2, plus the key step: each runtime proof is linked to the exact code line that causes it, then ranked by risk.

Mode 2 — how code is ingested

Input	First step	Note
Local folder path	Scanned in place, read-only.	No network at all — the strongest offline case. Working-tree secret scan only (no git history).
Repository URL	Shallow clone (--depth 1) to a temp workspace, scan, then delete.	The clone is the only networked moment; analysis is offline. Full clone optional for git-history secret scanning.

After ingestion both inputs are just a directory on disk, so every downstream step is identical. A *.gitignore* and a default skip-list (node_modules, venv, build output) keep the scan focused. AEGIS never writes into the user's folder; fix diffs are output separately.

Mode 2 — per-issue explanation format

Every finding in Mode 2 (and the code side of Mode 3) is reported to the user in the same five-part structure:

Field	Content
What	The vulnerability class and a one-line description.
Where	The exact file and line number, with the code shown.
Impact	What an attacker gains if it is abused; sensitivity of the affected data or function.
How it could be exploited	The attack path, at a conceptual level, tied to the evidence.
How to fix	A concrete remediation, drafted as a code diff where possible.

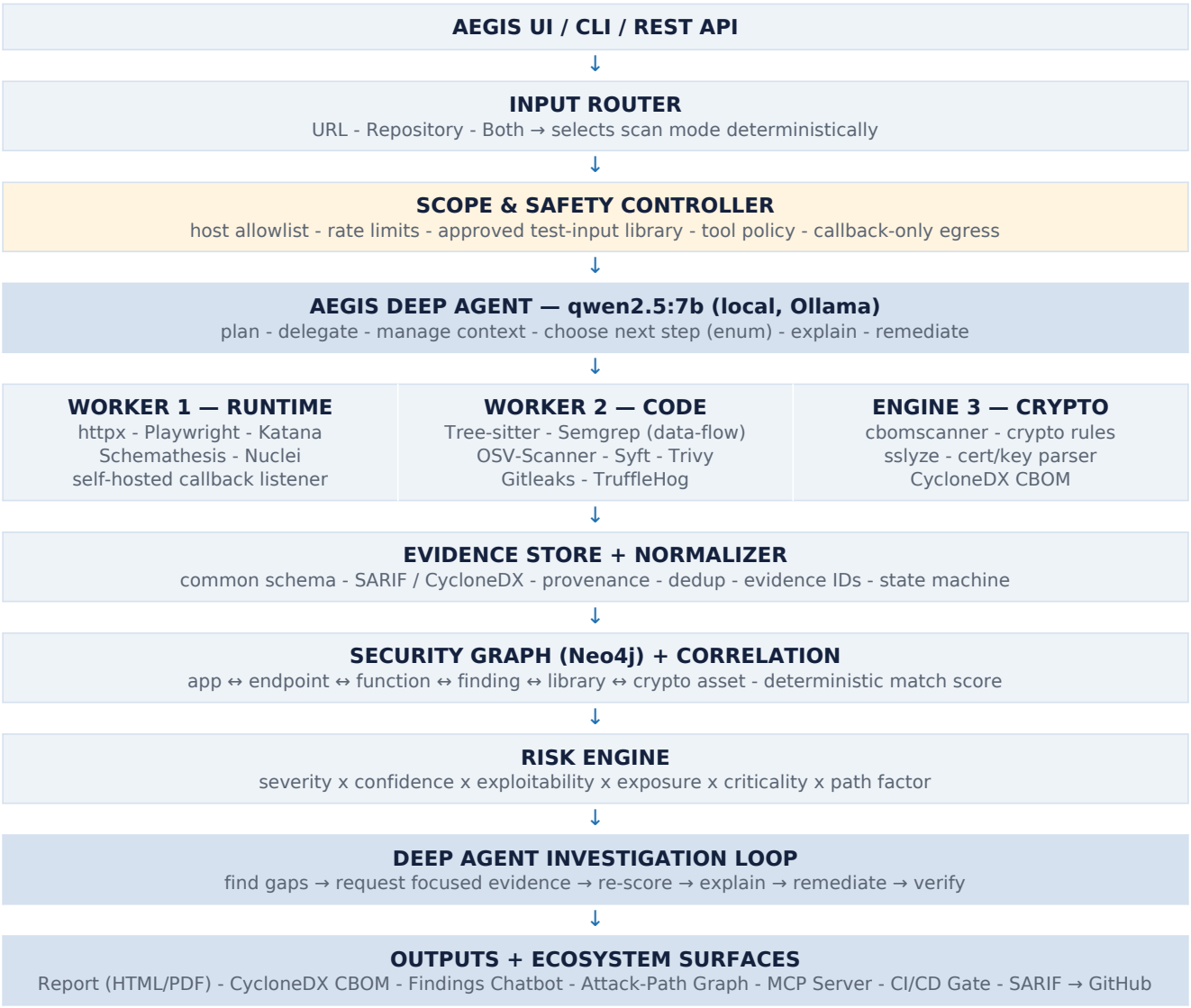
Mode 3 — why it is more than Modes 1 + 2 together

- Modes 1 and 2 running in parallel give two separate lists: a runtime list and a code list. Useful, but any toolbox does that.
- Mode 3 adds the **correlation step**: it matches each runtime finding to the handler function and line responsible, using the deterministic score in Section 9.
- Result: one issue, proven on the live app *and* located in code — the thing side-by-side scanners do not do.
- Correlation also feeds the risk engine, so a runtime-confirmed, code-located issue ranks above an isolated guess.

> **Scope honesty built into the modes:** Mode 1 is "all vulns it can *prove* at runtime" — not "all vulns." Code-only classes (vulnerable components, integrity, most logging issues) appear only in Mode 2 or Mode 3, because Worker 1 genuinely cannot see them.

3. System Architecture

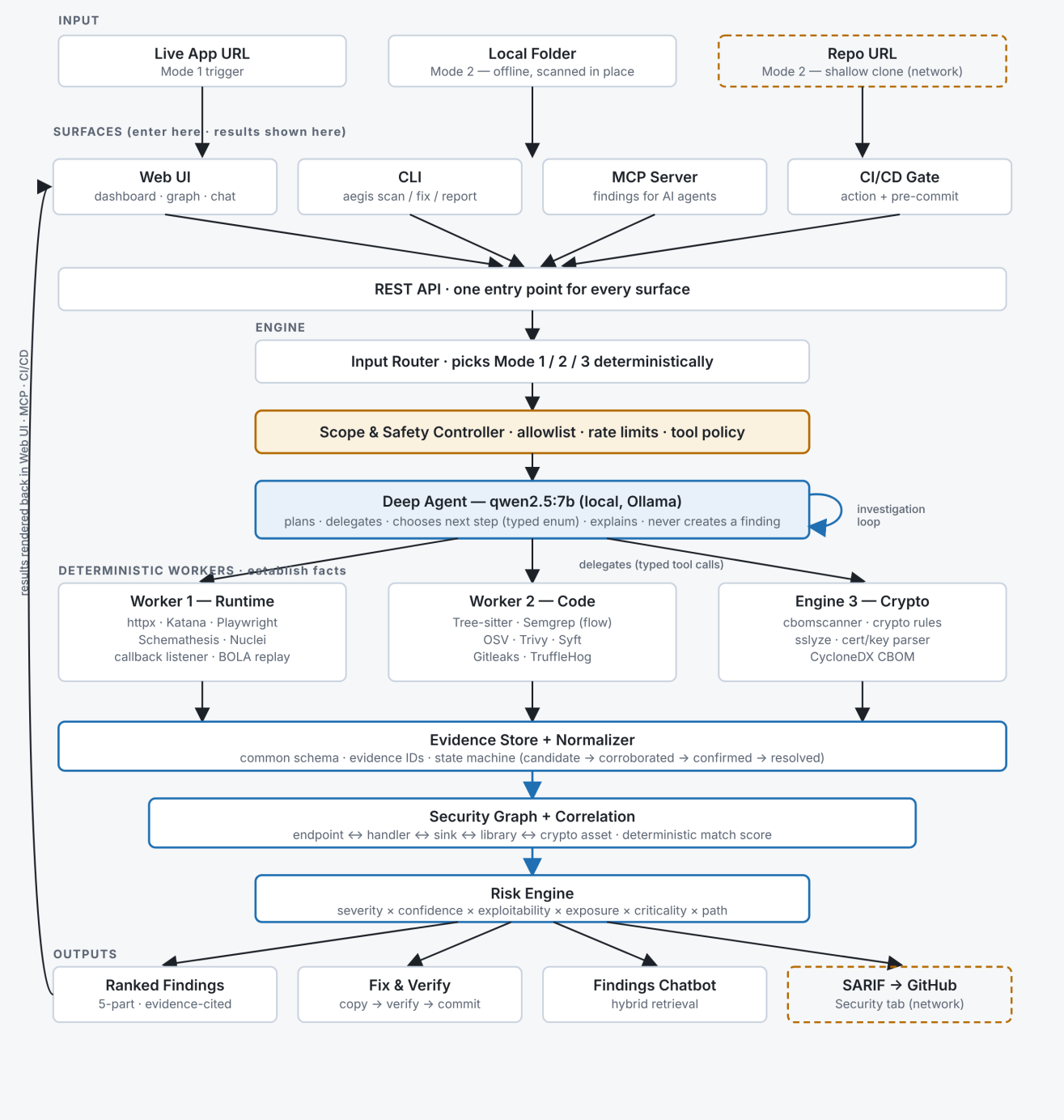
Three modular workers sit under one Deep Agent. Evidence, graph, correlation and risk are AEGIS-owned deterministic layers.



Security boundary: the Deep Agent never gets a shell. It can only call typed tool wrappers, and every call passes the Scope & Safety Controller first. The agent cannot widen scope, change test inputs or disable controls.

3B. Visual Flow — Full Component Map

The same architecture as a single top-to-bottom flow: inputs, the four surfaces, the REST API, the engine, the deterministic workers, the evidence core, and the outputs that return to the surfaces.



Accent outline = AEGIS-owned deterministic core (evidence store, correlation, risk). Dashed amber = the only connected-mode steps (cloning a repo, publishing SARIF to GitHub); everything else runs offline.

3A. Technology Stack (every component)

The complete toolchain, by layer: what each tool is, what it produces, and whether it runs offline. One engine, open-source tools, local model.

Layer	Tool / tech	Purpose	Offline
Model	qwen2.5:7b (Ollama)	Deep Agent reasoning, explanations, remediation drafts.	Yes
Model	nomic-embed-text (Ollama)	Embeddings for the findings chatbot.	Yes
Agent	LangChain / Deep Agents	Orchestration, sub-agents, typed tool wrappers.	Yes
W1 Runtime	httpx	Target validation, raw HTTP.	Yes
W1 Runtime	Katana	Crawl HTML/JS routes.	Yes
W1 Runtime	Playwright (Chromium)	Authenticated sessions, token capture.	Yes
W1 Runtime	Schemathesis	API conformance checks from OpenAPI.	Yes
W1 Runtime	Nuclei (pinned templates)	Known-issue and misconfig checks.	Yes*
W1 Runtime	Self-hosted callback listener	Out-of-band (SSRF) confirmation.	Yes
W2 Code	Tree-sitter	Parse code; route → handler extraction.	Yes
W2 Code	Semgrep / OpenGrep (local rules)	Static analysis with data-flow (source→sink).	Yes
W2 Code	OSV-Scanner (offline DB)	Known-vulnerable dependencies.	Yes*
W2 Code	Trivy (offline DB)	Deps, IaC, container, config issues.	Yes*
W2 Code	Syft	Component inventory / SBOM.	Yes
W2 Code	Gitleaks	Secret detection (working tree / history).	Yes
W2 Code	TruffleHog	Secret detection; live verify optional.	Yes*
Engine 3	cbomscanner + crypto rules	Crypto discovery → CycloneDX CBOM.	Yes
Engine 3	sslyze	Live TLS protocol/cipher/cert rating.	Yes
Data	Evidence store (SQLite)	Findings, provenance, states, evidence IDs.	Yes
Data	Neo4j (capped) / NetworkX	Security graph + correlation + attack paths.	Yes
Data	ChromaDB / FAISS	Vector index for the findings chatbot.	Yes
Surfaces	REST API / CLI	Core access, automation.	Yes
Surfaces	MCP server	Exposes findings to any AI agent.	Yes
Surfaces	CI/CD action + pre-commit	Build gate on new findings.	Yes
Surfaces	SARIF export	Publish to GitHub Code Scanning.	No**
Packaging	docker-compose	One-command bring-up of the whole stack.	Yes

* Runs offline against a pre-staged snapshot/DB; an optional connected mode refreshes feeds. ** SARIF publish to GitHub needs network by nature; the scan that produces it is offline.

4. Offline Runtime & Hardware Profile

AEGIS is built to run air-gapped on a mid-range laptop. Reference machine: 24 GB RAM, RTX 2050 with 4 GB VRAM.

Local model

Setting	Value	Why
Model	qwen2.5:7b (Q4, Ollama)	Best tool-calling and JSON output among models that fit the hardware.
Context (num_ctx)	8192	Keeps the KV cache small so most layers stay on the 4 GB GPU.
Temperature	0.1	Repeatable decisions and demo runs.
Output	JSON schema / enum only	A 7B model is reliable only when constrained; free-form planning is disallowed.
Endpoint	OpenAI-compatible (Ollama)	Model is swappable by config — a larger model on stronger hardware needs no code change.

Memory budget (approximate)

Component	Budget	Notes
qwen2.5:7b	~5 GB	Mostly GPU, small spill to RAM.
Scanners + demo app + listener	~4 GB	Tools run one stage at a time, not all at once.
Neo4j	≤1 GB heap	Capped heap. Fallback: NetworkX in-process.
Playwright	~0.5 GB	One browser instance, closed after login capture.
OS + UI + headroom	rest	Heavy JVM tools (SonarQube, Joern) are deliberately excluded from the MVP.

Pre-staged offline assets

- Pinned Nuclei template snapshot (update checks disabled).
- Offline vulnerability databases for OSV-Scanner and Trivy; the DB date is printed on every report.
- Local Semgrep/OpenGrep rule packs including AEGIS custom rules; telemetry off.
- Self-hosted callback listener on the same Docker network as the target, so out-of-band checks never reach the internet.
- Everything ships as one **docker-compose**: AEGIS, Ollama, Neo4j, listener, demo app.

Optional connected mode	What it adds
Feed update	Refresh vulnerability DBs and Nuclei templates, then return to offline.
Secret verification	Live verification upgrades a secret from candidate to confirmed. Off by default because it contacts providers.

5. The Deep Agent

The Deep Agent acts as the Security Lead: it decides what to look at next and explains what the evidence means. It never decides what is true.

The agent MAY	The agent MAY NOT
Plan the task sequence for the detected mode	Create, edit or delete a finding
Delegate work to Worker 1, Worker 2, Engine 3	Run arbitrary commands or choose test inputs
Choose the next investigation step from a fixed enum	Expand scope or change rate limits
Write explanations that cite evidence IDs	Cite evidence IDs that do not exist
Draft remediation diffs and request verification	Mark a finding resolved without a passing re-test

Constrained decision step

For each open finding, rules handle the obvious cases first. The model is consulted only when several next steps are valid, and it must answer in this schema:

```
{ "finding_id": "F-0012",
  "next_step": "request_source_evidence" | "request_runtime_retest" | "request_crypto_scan"
            | "link_and_score" | "mark_sufficient" | "escalate_to_human",
  "target": { "route": "GET /api/accounts/{id}", "function": "get_account" },
  "reason_evidence_ids": ["E-0041", "E-0057"] }
```

Invalid JSON, an unknown enum value or an unknown evidence ID is rejected and retried once; a second failure falls back to the rule-based default.

Evidence-ID contract (anti-hallucination)

- Every explanation is returned as a list of claims; each claim must reference one or more evidence IDs.
- A validator checks every ID against the evidence store before anything reaches the UI or report.
- In the UI, each sentence is clickable and opens the raw request/response or code line behind it.
- A claim with no valid evidence is dropped and logged — the model cannot talk a finding into existence.

Context management

- The agent receives normalized fields only: finding type, route, function, file:line, state, scores, evidence IDs.
- Raw HTTP bodies, code comments and file contents stay in the evidence store, never in the prompt. This keeps the 8K context sufficient and blocks prompt injection from target content.
- Each sub-task gets a fresh, minimal context instead of one growing transcript.

6. Worker 1 — Runtime Analysis

Establishes facts about the deployed application through controlled, authorized interaction. All activity is bounded by the Scope & Safety Controller.

#	Stage	What it does	Tools
01	Validate	Confirm target URL and that it is inside the authorized scope allowlist; load scan policy.	httpx
02	Discover	Enumerate reachable routes from HTML and JS; parse OpenAPI/Swagger if the app publishes one.	Katana, OpenAPI parser
03	Authenticate	Sign in with each supplied authorized test identity and capture its session token.	Playwright
04	Model	Build a map of endpoint x method x parameter x identity x expected response.	AEGIS
05	Check	Run authorized conformance checks for access-control, server-side request, injection and misconfiguration weaknesses against the map.	AEGIS, Nuclei, Schemathesis, callback listener
06	Record	Capture request, response, status, timing and headers as reproducible evidence.	AEGIS

Access-control checks (the flagship capability)

Broken object-level authorization is the hardest class for generic tools because it is application-specific. AEGIS handles it with an authorized differential method: the same request is replayed under two supplied test identities and the responses are compared. If identity B receives identity A’s data, that difference is the evidence. This is an AEGIS-built check and a key differentiator.

Safety: Worker 1 only ever runs against a target on the host allowlist, using the approved test-input library and supplied test accounts, under rate limits. It cannot be pointed at an arbitrary host by the model or by target content.

7. Worker 2 — Code Analysis

Explains why a runtime fact is true and finds issues that never surface at runtime. Produces file/function/line evidence.

Capability	Tool	Evidence produced
Repository + language inventory	Syft, Tree-sitter	Components, languages, SBOM
Route → handler extraction	Tree-sitter, framework rules	Endpoint-to-function map (basis for correlation)
Static analysis with data-flow	Semgrep / OpenGrep	Source-to-sink path with file:line
Known-vulnerable dependencies	OSV-Scanner, Trivy (offline DB)	Package, version, advisory ID
Exposed secrets	Gitleaks; TruffleHog (optional verify)	File:line; candidate or confirmed
IaC / container misconfig	Trivy	Misconfigured resource + rule

8. Engine 3 — Cryptographic Risk

A fixed pipeline that inventories cryptography and rates its risk, now scoped to run on the reference laptop (no SonarQube/JVM dependency).

Stage	Action	Tool
Discover	Find crypto usage in source and dependencies.	cbomscanner, Semgrep crypto rules
Inventory	Emit a CycloneDX Cryptography Bill of Materials (CBOM).	AEGIS (CycloneDX 1.6+)
Classical risk	Flag weak algorithms, short keys, weak modes, bad randomness.	AEGIS rules
Quantum risk	Flag harvest-now-decrypt-later exposure (RSA/ECC in transit/storage).	AEGIS rules
TLS posture	Rate the live endpoint’s protocols, ciphers and certificates.	sslyze
Correlate	Link a live TLS/crypto observation to the source config that sets it.	AEGIS

Scope honesty: the crypto engine ships in the MVP as CBOM + classical/quantum flags + one correlated TLS finding. A full compliance mapping (e.g. regulatory profiles) is roadmap, not claimed as built.

9. Correlation — the Core Differentiator

Correlation turns separate findings into one explained issue: a runtime fact, its root cause in code, and the crypto around it. It is fully deterministic — no model is involved in deciding a link.

Step 1 — Build the endpoint↔handler map

Worker 2 extracts routes from the framework. The MVP supports **one framework (Flask)** so the mapping is reliable on stage; the method generalizes to Express and Spring later.

Source	Extracted	Normalized key
@app.route('/api/accounts/<id>')	path + method + handler name	GET /api/accounts/{id} → get_account
Runtime discovery	observed path + method	GET /api/accounts/{id}

Both sides are normalized the same way: lowercase method, path template with *{param}* placeholders, trailing slash stripped. A match on the normalized key links the runtime finding to the handler function.

Step 2 — Score the link (deterministic)

Signal	Contribution
Normalized route template matches handler route	+0.5
HTTP method matches	+0.2
Data-flow path confirms handler reaches the flagged sink	+0.3
(score is capped at 1.0; only edges ≥ 0.70 are kept as correlated)	

The score is stored on the graph edge and shown in the UI. A judge asking “how is correlation computed?” gets this table, not an arrow.

Step 3 — Walk the graph

With links scored, one query returns a full issue: endpoint → handler → data-flow path → (optional) weak dependency or crypto asset. That chain is what AEGIS reports instead of six disconnected alerts.

10. Risk Engine

Every correlated issue gets one transparent score. The formula is published in the report so a reviewer can recompute it by hand.

```
Risk = Severity x Confidence x (1 + Exploitability + Exposure + Criticality + PathFactor) / Kmax
```

Factor	Range	Source
Severity	0.1 – 1.0	Base weakness class weight (CVSS-aligned).
Confidence	0.1 – 1.0	Evidence state: candidate 0.4, corroborated 0.7, confirmed 1.0.
Exploitability	0 – 1.0	Runtime-proven = high; source-only = lower.
Exposure	0 – 1.0	Unauthenticated / internet-facing raises it.
Criticality	0 – 1.0	Sensitivity of the data or function behind the endpoint.
PathFactor	0 – 1.0	Graph connectivity: an issue on an attack path to sensitive data scores higher.

Kmax normalizes the bracket so Risk stays in 0–1. Because Confidence and PathFactor are inputs, a runtime-confirmed, correlated issue outranks an isolated source-only guess — correlation visibly changes priority.

11. Evidence State Machine

Every finding has exactly one state, and the state is what Confidence reads from. Nothing is “confirmed” without reproducible proof.

CANDIDATE	**CORROBORATED**	**CONFIRMED**	**RESOLVED**
one tool, one signal	two independent signals	reproduced / proven	fix re-tested, passes

A source-only pattern is a **candidate**. A runtime difference that reproduces is **confirmed**. If the model claims something the evidence does not support, the claim is dropped — the state never advances on the model’s say-so.

12. End-to-End Walkthrough (Demo Script)

A single access-control issue carried from runtime proof to code root cause to verified fix. This is the story to show judges.

#	What happens	State / output
1	Input: demo app URL + its repository. Router selects combined mode.	mode = both
2	Worker 1 authenticates two supplied test identities and maps the endpoints.	endpoint map
3	Differential replay: identity B receives identity A's record on GET /api/accounts/{id}.	F-0012 CONFIRMED (runtime)
4	Deep Agent decides next_step = request_source_evidence for that route.	enum decision, logged
5	Worker 2 maps the route to get_account() and data-flow shows no owner check before the DB read.	root cause, file:line
6	Correlation links F-0012 to get_account(): 0.5 + 0.2 + 0.3 = 1.0.	edge score 1.0
7	Risk engine ranks it top: confirmed + unauth + sensitive data + on path.	Risk ≈ 0.9
8	Agent drafts the owner-check fix as a diff; explanation cites E-0041, E-0057.	patch + cited claims
9	Fix applied to a pre-staged branch; differential replay re-runs; B is now denied.	F-0012 RESOLVED

Why an agent, not a pipeline? Step 4 is the answer: the agent saw a confirmed runtime fact with no root cause and chose to go get source evidence, which moved the finding from confirmed to explained-and-fixed. That is an investigation, not a fixed script.

Demo prerequisites to stage in advance

- A small **single-framework (Flask)** demo app you control, with a handful of intentional, well-understood weaknesses.
- Two pre-provisioned test identities for the differential access-control check.
- A self-hosted callback listener on the Docker network for out-of-band checks.
- A **pre-built patched branch** to switch to for the live fix — do not edit and redeploy live on stage.
- Run once offline before the review and save the report; keep Wi-Fi off during the demo as the headline moment.

12B. Click-to-Fix & Verify Loop

A code finding is not the end — the user can apply the drafted fix and have AEGIS confirm it. Fixes are proven on a copy first, then committed to the original only with the user's approval. AEGIS never silently edits the user's code.

Three stages

1. FIX on COPY apply diff to a working copy	2. VERIFY on copy re-scan: pattern gone + tests pass?	3. COMMIT to original user approves: patch / apply / PR

Only a fix that passes stage 2 is offered for stage 3. A fix that fails the re-scan stays open, with the reason shown — it never reaches the real code.

Commit option	What happens	Best for
Download patch	AEGIS outputs the .diff; user applies it with git apply.	Full manual control
Apply to original	One click writes the verified diff into the real file.	Local folder mode
Open a Pull Request	Pushes a branch + PR with the fix (connected mode).	Repo mode / team review

In repo mode the original is a remote repo, so the fix lands as a **Pull Request**, never a silent push. In local-folder mode it writes to the file on disk after approval.

Example 1 — SQL injection (Semgrep data-flow)

```
# BEFORE app/users.py:42 (F-0007, E-0101)
q = "SELECT * FROM users WHERE name = '" + name + "'"
db.execute(q)  # untrusted input in SQL string

# AFTER (fix applied to copy)
q = "SELECT * FROM users WHERE name = %s"
db.execute(q, (name,))  # parameterized
```

Re-scan: data-flow path broken, 1 finding → 0. **Verdict: RESOLVED — injection path no longer detected (tests 12/12).**

Example 2 — Hardcoded secret (Gitleaks)

```
# BEFORE config.py:8 (F-0019, E-0133)
API_KEY = "sk_live_****************"  # secret in source

# AFTER (fix applied to copy)
import os
API_KEY = os.environ["API_KEY"]  # moved out of source
```

Re-scan: 1 secret → 0 in working tree. **Verdict: RESOLVED (working tree).** Warning: the key was already committed — rotate it; removing it from code does not un-leak the old value.

> **Honesty on "fixed":** a static re-scan proves the *pattern is cleared and tests pass* — not that the code is formally correct. In Mode 3, Worker 1 then re-tests the live app, so an access-control fix shows "pattern gone in code *and* exploit no longer works live" — the strongest proof.

12C. Conversational Query — Chat with your Findings

After a scan, the user can ask questions about the results in plain English. This is a **hybrid retrieval** system, not pure RAG — because findings are structured records, and most questions are filters and lookups, not semantic search.

Why hybrid, not pure RAG

A question like “show my critical findings” is a field filter, not a similarity match. Pure vector search would approximate it and risk miscounting or missing findings. So AEGIS routes each question down the path that answers it exactly.

Question type	Retrieval path	Example
Filter / count / lookup	Structured query over the evidence store (SQLite / Neo4j)	“list confirmed A01 findings” · “how many criticals?” · “what’s in auth.py?”
Explain / why / how	Vector RAG over each finding’s text (what / impact / fix)	“explain the account leak simply” · “why is this exploitable?”

Components (all local, offline)

Role	Choice	Note
Embeddings	nomic-embed-text (Ollama)	~270 MB; do not use the 7B for embeddings.
Vector store	ChromaDB / FAISS (local)	One embedded document per finding, built from its five-part text.
Structured store	Evidence store (reused)	Already exists — no new data layer.
Generator	qwen2.5:7b	Same model as the rest of AEGIS.
Router	Small classifier / rules	Sends each question to the structured or vector path.

Same evidence contract as the rest of AEGIS

- Every answer cites finding IDs and evidence IDs — identical to the reports.
- The model answers **only** from retrieved findings. If nothing retrieves, it says “no finding matches” — it does not improvise security advice.
- It cannot change a finding’s state or invent a vulnerability the scanners did not produce.
- Net effect: not “chat with your data” but **chat with your evidence** — consistent with the whole platform.

> **Build order:** this is built *after* the core scan and correlation work. It reuses the evidence store and adds no new findings logic, so it is low-risk — but it must never become the feature that delays correlation.

12D. Ecosystem & Integrations

AEGIS is one engine with many surfaces. It meets developers where they already work instead of forcing them into a new dashboard — the real edge over a standalone scanner. All four surfaces read from the same evidence store and honor the same citation contract.

THE ENGINE	THE SURFACES
scan - correlate - rank - evidence store	MCP - CI/CD - Graph - SARIF - Chatbot - API/CLI

1. MCP Server — findings for any AI agent

Exposes the evidence store as a Model Context Protocol server, so Claude, Cursor or any MCP-aware agent can query findings directly (“what are the critical issues in this repo?”) with the same evidence-ID answers. This is the core of the ecosystem claim: AEGIS becomes a security data source the whole agent world can plug into. Runs locally.

2. CI/CD Gate — security in the pipeline

A GitHub Action and a pre-commit hook run `aegis scan` automatically and fail the build when new findings exceed a severity threshold. This turns AEGIS from a tool into infrastructure: bad code is blocked at the exact line before it merges. Reuses the CLI; runs offline in a runner.

3. Attack-Path Graph — correlation made visible

An interactive view of the security graph: endpoint → handler → sink → dependency/crypto asset, with the correlation score on each edge. It makes the differentiator (correlation) something judges can see and click, not an abstraction. Backed by Neo4j/NetworkX, rendered in the UI.

4. SARIF → GitHub Code Scanning — standards interop

Findings export to SARIF, the standard JSON format for security results. Uploaded to GitHub, they appear natively in the repo’s Security tab and as inline comments on the exact lines in a pull request — no custom UI required. This is a connected-mode feature; the scan that produces it is offline.

> **The ecosystem pitch, in one line:** offline, evidence-first scanning that shows up in your IDE, your CI/CD pipeline, your AI agent (MCP) and GitHub (SARIF) — all answering from one evidence store with one citation contract.

12E. Surfaces Specification

One engine, many surfaces. The CLI does the work; the Web UI, MCP server and CI/CD gate are front doors to the same scan command and evidence store.

CLI — the core interface

Command	What it does
`aegis scan --url <u>`	Mode 1 — runtime scan of a live application.
`aegis scan --repo <url> | --folder <p>`	Mode 2 — code + cryptography analysis.
`aegis scan --url <u> --folder <p>`	Mode 3 — both, plus cross-layer correlation.
`aegis fix <id>`	Apply the drafted fix to a copy, verify by re-scan, show the result.
`aegis fix <id> --commit=patch|apply|pr`	Commit a verified fix: patch, write to file, or open a PR.
`aegis report --format=html|sarif|cbom|json`	Export results.
`aegis ask "<question>"`	Query findings from the terminal.
`aegis serve`	Start the REST API, Web UI and MCP server.

Flags: `--mode`, `--severity-threshold`, `--offline`, `--identities <file>`, `--output <dir>`. Exit codes: 0 clean, 1 findings above threshold (fails a CI build), 2 error.

Web UI — interactive review

Screen	What it provides
New Scan	URL / folder / both; live progress by worker.
Findings dashboard	Ranked list; filter by severity, OWASP, mode, state; state badge per finding.
Finding detail	Five-part view; evidence IDs click to raw request-response or code line.
Click-to-Fix panel	Diff, Apply & Verify, before/after re-scan result.
Attack-path graph	endpoint → handler → sink → crypto, edges labeled with correlation score.
Chatbot panel	Plain-English questions; answers cite finding and evidence IDs.
Reports	Download HTML, SARIF or CBOM.

12E. Surfaces Specification (continued)

MCP server — findings for any AI agent

Read-only, local, same evidence contract — agents can read findings but cannot invent or change them.

MCP tool	Returns
`list_findings(filters)`	Findings filtered by severity, category or state.
`get_finding(id)`	Full detail for one finding, with evidence IDs.
`get_evidence(id)`	The raw proof behind a claim.
`get_attack_paths()`	Correlated chains from the security graph.
`get_cbom()`	The cryptography bill of materials.
`scan_status()`	Current or last scan state.

CI/CD gate — security in the pipeline

Form	Behavior
GitHub Action	On push/PR: runs the scan, uploads SARIF to the Security tab, fails the build if findings exceed the threshold; inline PR comments on exact lines.
Pre-commit hook	Fast scan before a commit; blocks locally on a serious finding (quick scan, not the full suite).

Configuration: a `.aegis.yml` file sets the severity threshold, the scan mode, and which OWASP categories gate the build versus merely warn.

> **The through-line:** the CLI does everything; the Web UI, MCP server and CI/CD gate are different front doors to the same scan and the same evidence store — “one engine, many surfaces” made concrete.

12A. OWASP Top 10 Coverage (2021)

Which worker or engine finds each category, the signal it detects it from, and the honest confidence. Owner also tells you the mode it appears in.

OWASP category	Owner	Detection signal	Confidence
A01 Broken Access Control	W1 + W2	Same request replayed under two test identities; responses differ. W2 shows missing owner check.	Strong
A02 Cryptographic Failures	Engine 3 + W1	Weak algorithm/key/mode in source + CBOM; sslyze rates live TLS.	Strong
A03 Injection	W1 + W2	Unsafe-handling response signatures; Semgrep source-to-sink data-flow.	Strong
A04 Insecure Design	W2	Risky design patterns only; architecture itself is not machine-judgeable.	Weak (human review)
A05 Security Misconfiguration	W1 + W2	Missing headers, verbose errors, exposed endpoints; Trivy on IaC/containers.	Strong
A06 Vulnerable Components	W2	Syft inventory matched to offline OSV/Trivy advisory DB.	Strong
A07 Auth Failures	W1 + W2	Session/token behavior observed live; auth code path reviewed.	Medium
A08 Integrity Failures	W2	Untrusted deps, unpinned versions, unsafe deserialization in code/CI.	Medium
A09 Logging/Monitoring	W2	Sensitive handlers with no logging call. Absence is not observable live.	Weak (human review)
A10 SSRF	W1	URL-taking parameters checked; self-hosted callback listener records out-of-band hit.	Strong

The honest headline: 7 categories are strongly covered, 2 are medium, and 2 (A04, A09) are flagged for human review rather than guessed. The two gaps are hard for every automated tool — stating them is what makes the other eight credible.

Reading this by mode

- **Mode 1 (URL)** reaches A01, A03, A05, A07, A10 — the runtime-provable classes.
- **Mode 2 (repo)** reaches A02, A04, A06, A08, A09 and the code root causes — including the classes that never appear at runtime.
- **Mode 3 (both)** is where A01 and A03 become strongest, because the runtime proof and the code cause are correlated into one finding.

13. Honest Build Scope

State this split up front. It pre-empts the first question judges ask: “what is actually running?”

Built for MVP (demo)	Extended (if time)	Roadmap (stated, not claimed)
Combined-mode run on Flask demo app	Express / Spring route extraction	Multi-language repos at scale
Differential access-control check	More weakness classes	Full IAST-style instrumentation
Semgrep data-flow, OSV/Trivy offline	Joern code-property-graph paths	Distributed scanning
CBOM + classical/quantum flags + 1 TLS link	Broader crypto correlation	Regulatory compliance profiles
Deterministic correlation + risk	Richer graph queries	Learned risk weighting
Evidence IDs + verify loop	Verified-secret mode	Team workflow / ticketing
SARIF + MCP server + CI/CD gate	Attack-path graph UI, chatbot	IDE extension, codebase-wide chat

14. Differentiation

Existing approach	What AEGIS adds
SAST platforms (Semgrep, etc.)	Pairs each source finding with runtime proof and a risk that reflects it.
DAST / runtime scanners	Carries every runtime fact down to the exact line of code.
AI security agents (cloud)	Fully offline; the model may not invent findings; every claim is evidence-linked.
Crypto / CBOM tools	Crypto risk sits in the same graph and risk model as the AppSec findings.

One-line claim: **offline, evidence-gated, cross-layer investigation** — runtime proof, code root cause and crypto risk, correlated and ranked, on a laptop with no internet.

15. Known Limitations (state them before judges do)

- Local 7B model is weaker at planning than cloud models; mitigated by enum-constrained decisions and rule-first handling.
- Correlation is reliable for one framework in the MVP; other frameworks are extended scope.
- Offline vulnerability data is only as fresh as the last snapshot; the DB date is printed on every report.
- Secret verification needs network and is off by default; unverified secrets stay candidate.
- AEGIS runs only against targets you are authorized to test; scope enforcement is a control, not a licence.

Bottom line. AEGIS v2.1 is a focused, offline, evidence-first system: deterministic tools find and prove, a constrained local agent investigates and explains, and one graph ties runtime, code and cryptography into a single ranked, auditable result.

16. Team Build Plan (two people)

The fastest path for a two-person team: agree the data contract first, build one thin vertical slice together, then split cleanly.

Step 0 — do together first

- **Freeze the evidence JSON schema.** One finding = one object: id, type, severity, state, route, file, line, evidence_ids, what, impact, fix. The scanner writes it; the UI and agent read it.
- **Pick the demo app and stack.** One small Flask app with a handful of intentional, well-understood weaknesses; Python backend, Ollama endpoint, SQLite evidence store, simple web UI; one repo, two branches.

The split

Person A — Engine	Person B — Agent + UI + Demo
Worker 2 (Semgrep + Gitleaks) → evidence store	Deep Agent on qwen2.5:7b (has the model)
Worker 1 differential access-control check	Web UI: dashboard, detail, attack-path graph
Correlation (route→handler + score)	Demo Flask app with planted weaknesses
Evidence store (SQLite) + findings JSON	CLI entry point (aegis scan)

Build order — get a vertical slice working fast

Day	Milestone
1	Schema frozen. Person B ships the demo app + a sample findings.json. Person A starts Worker 2. Person B builds the dashboard against the sample JSON.
2	Real Semgrep finding replaces the sample. Agent explains one finding. Milestone: scan → real finding → shown in UI with explanation.
3	Person A adds the runtime access-control check + correlation. Person B adds the attack-path graph + fix-and-verify.
4	Mode 3 end-to-end (the walkthrough). Polish the demo script.
Buffer	One ecosystem surface (MCP server is cheapest) only if the core is solid.

> **The rule that wins:** get the vertical slice working by end of Day 2 — scan one real bug, show it in the UI, agent explains it. Everything after is adding finding types and surfaces to a thing that already works.

Cut order if time runs short

- **Keep:** access-control + SQL-injection + secret detection, correlation, UI, agent explain, fix-and-verify.
- **Drop first (state as roadmap):** crypto beyond CBOM, chatbot, SARIF, CI gate, broad OWASP coverage.